Data Transformation and Cleansing with Python and Pandas

Ensuring high-quality data is a critical task in any data analysis or data science project. Dirty or messy data can lead to misleading analysis or produce inaccurate models. This exercise will elaborate on how Python's Pandas library can be used for data transformation and cleansing, taking a deeper dive with more detailed examples.

1. Setup and Initialization

Before diving into the practical parts, let's setup our environment:

```
import pandas as pd
import numpy as np
```

2. Creating a Sample Dataframe

Our example dataframe mimics a dataset containing personal details and some anomalies:

```
data = {
   'Name': ['Alice', 'Bob', 'Charlie', np.nan, 'Eve', 'Frank', 'Grace', 'Alice'],
   'Age': [25, np.nan, 35, 40, 50, 'Unknown', 30, 25],
   'Gender': ['Female', 'Male', np.nan, 'Female', 'Other', 'Male', 'Female', 'Female'],
   'Salary': [50000, 52000, 55000, np.nan, 48000, 'N/A', 51000, 50000],
   'Email': ['alice@email.com', 'bob@email', 'charlie@email.com', '', 'eve@email.com', '',
'grace@email.com', 'alice@email.com']
}

df = pd.DataFrame(data)
print(df)
```

2.1

Data Sets (Data Frames) can also be imported from files using panda.read_csv.

Panda.DataFrame transforms the data into a Data Frame (Table) that Python can use.

```
df_csv = pd.read_csv("Sample_Data_Frame.csv",sep=',')


print(df_csv)
```

3. Identifying and Handling Missing Values

3.1. Identification

In the code below all NaN vales are counted to determine the number of missing values and returned directly to the screen. Note that empty cells and N/A values are not included in the count. Note isnull looks for integers (NaN indicates a value that is “not a number”)

```
print(df_csv.isnull().sum())
```

3.2. Filling Missing Values

Using Default Values:

In the code below the Name and Gender fields will have their NaN values exchanged for Unknown. The resulting values are written to a new dataframe called df_filled. The df_filled is then shown to the screen using the print command.

```
df_filled = df_csv.fillna({'Name': 'Unknown', 'Gender': 'Unknown'})
print(df_filled)
```

The code below will replace the values Unknown in the Age field and N/A in the Salary field with NaN. The results will replace inplace within the df dataframe.

```
df.replace({'Age': 'Unknown', 'Salary': 'N/A'}, np.nan, inplace=True)
```

Check the differences by using the print command again.

```
print(df)
```

Using Mean and Median

Calculate the median age of the dataframe and set the age_median variable.

The value should be 32.5 .

```
age_median = df['Age'].median(skipna=True)

print(age_median)
```

The mean of a number may be calculated and set for any missing values as below. The value should be 51000.

```
salary_mean = df['Salary'].mean(skipna=True)

print(salary_mean)
```

These prior set values can be replaced in the dataframe in place of NaN values

```
df['Age'].fillna(age_median, inplace=True)

df['Salary'].fillna(salary_mean, inplace=True)

print(df)
```

4.1. Finding and Replacing Incorrect Values

The replace function can be employed: inplace=True will overwrite the existing data.

df.replace({'columnName': 'Value'}, np.nan, inplace=True) – This is an example

4.2. Ensuring Data Type Consistency

Converting columns to the right data types ensures consistency:

The code below converts the Age to an integer and the Salary to a float.

Use the print command to view the change made.

```
df['Age'] = df['Age'].astype(int)
df['Salary'] = df['Salary'].astype(float)

print(df)
```

5. Data Transformation

5.1. Normalization

Normalization ensures data falls within a similar scale:

One way of normalizing the data is to create a scale between 0 and 1 for each value. In the code below the scale runs from the minimum value of Salary to the maximum. Each salary is then a percentage of the difference. 0.000 is given to the minimum value and 1 to the maximum value.

```
df['Salary'] = (df['Salary'] - df['Salary'].min()) / (df['Salary'].max() - df['Salary'].min())

df['Salary']
```

The same formula is used for the Age normalization process.

```
df['Age_Normaized'] = (df['Age'] - df['Age'].min()) / (df['Age'].max() - df['Age'].min())

df['Age_Normaized']
```

5.2. Binning

Grouping continuous values:

The code below creates the edges of the bins for the dataframe. The number of labels must be one less than the number of bins. For instance the first two bin values are 0 and 30. So the gap between them will have the first of the labels.

```
bins = [0, 30, 40, 50, 60]

labels = ['20s', '30s', '40s', '50s']
```

The option for right=False instructs the function to include the value as part of the latter label bin. The code below adds an age group field to the data frame with the correct bin label.

```
df['Age Group'] = pd.cut(df['Age'], bins=bins, labels=labels, right=False)

print(df)
```

The finished code looks like this:

```
bins = [0, 30, 40, 50, 60]

labels = ['20s', '30s', '40s', '50s']

df['Age Group'] = pd.cut(df['Age'], bins=bins, labels=labels, right=False)

print(df)
```

5.3. One-hot Encoding

Converting categorical variables:

The following code will create categorical variables (allowing some statistical analysis like correlations to be run) for the Gender columns, then apply the concat function to add the extra field.

```
gender_encoded = pd.get_dummies(df['Gender'], prefix='Gender')

df = pd.concat([df, gender_encoded], axis=1)

print(df)
```

6. Text Data Cleaning

6.1. Stripping Whitespaces

We can check the lengths of the email addresses using the len function of pandas

```
df['Length_Email'] = df['Email'].str.len()
```

In the email address example the email addresses are well formatted without extra spaces. Use the strip() function to remove the extra spaces. The code below will add a column before and after the strip function has run to show the differences.

```
df['Length_Name'] = df['Name'].str.len()

print(df)

df['Name'] = df['Name'].str.strip()

df['Length_NameAfterStrip'] = df['Name'].str.len()

print(df)
```

6.2. Validating Email

Checking for '@' in the email column:

```
def is_valid_email(email):
    if '@' in email:
        return email
    else:
        return np.nan

df['Email'] = df['Email'].apply(is_valid_email)

print(df)
```

6.3. Handling Text Case

Ensuring consistency in case:

```
df['NameTitle'] = df['Name'].str.title()
print(df['NameTitle'])

df['NameUpper'] = df['Name'].str.upper()
print(df['NameUpper'])

df['NameLower'] = df['Name'].str.lower()
print(df['NameLower'])
```

7. Deduplication

Removing duplicates ensures accuracy:

```
df.drop_duplicates(keep='first', inplace=True)
print(df)
```

8. Aggregating Data

Aggregating data can help summarize information:

```
print(df.groupby('Gender').agg({'Age': ['mean', 'median','std','var','sum','first','last','min','max']}))
print(df.groupby('Gender').agg({'Age': ['count', 'mean', 'nunique','unique']}))
```

9. Reshaping and Pivoting

9.1. Melting

Converting columns into rows:

```
df_melted = pd.melt(df, id_vars=['Name'], value_vars=['Age', 'Salary'])
print(df_melted)
```

9.2. Pivoting

The opposite of melting; converting rows to columns:

```
df_pivoted = df.pivot(index='Name', columns='Gender', values='Salary')
print(df_pivoted)
```

10. Handling Date and Time

If we had a datetime column, we could extract specific components:

 # Example: '2021-08-10 15:30:45'

```
date_example_df = pd.DataFrame({
    'Name': ['Alice', 'Bob Newhart'],
    'DOB_String': ['2021-08-10 15:30:45','2021-08-10 15:30:45']
})
date_example_df['DOB_DateTime'] = pd.to_datetime(date_example_df['DOB_String'])

print(date_example_df)

date_example_df['Year'] = date_example_df['DOB_DateTime'].dt.year
date_example_df['Month'] = date_example_df['DOB_DateTime'].dt.month
date_example_df['day'] = date_example_df['DOB_DateTime'].dt.day
print(date_example_df)


date_example_df['hour'] = date_example_df['DOB_DateTime'].dt.hour
date_example_df['minute'] = date_example_df['DOB_DateTime'].dt.minute
date_example_df['second'] = date_example_df['DOB_DateTime'].dt.second
print(date_example_df)
```

11. Conclusion

This comprehensive exercise showcased the vast capabilities of the Pandas library in data transformation and cleansing. Regularly employing these techniques ensures data integrity and lays a solid foundation for subsequent analysis or modelling tasks. As you continue to work with different datasets, remember that understanding your data is crucial, and the techniques you apply should be based on the specific challenges and requirements of each dataset.

Cocat datasets

```
data1= {
    'Name': ['Alice', 'Bob Newhart', '     Charlie', np.nan, '          Eve', 'Frank', 'Grace', 'Alice'],
    'Age': [25, np.nan, 35, 40, 50, 'Unknown', 30, 25],
    'Gender': ['Female', 'Male', np.nan, 'Female', 'Other', 'Male', 'Female', 'Female'],
    'Salary': [50000, 52000, 55000, np.nan, 48000, 'N/A', 51000, 50000],
    'Email': ['alice@email.com', 'bob@email', 'charlie--email.com', '', 'eve@email.com', '',
'grace@email.com', 'alice@email.com']
}

df1 = pd.DataFrame(data1)
df1['DataGroup'] = 'Company_A'
print(df1)

data2= {
    'Name': ['Graham', 'Simon', 'John', 'Fred', 'Francis'],
    'Age': [85, 76, 25, 43, 62],
    'Gender': ['Male', 'Male', np.nan, 'Male','Female'],
    'Salary': [12000, 52000, 55000, 48000, 50000],
    'Email': ['g_able@email.com', 'simon@here.com', 'jj@email.com', 'fred_clampett@email.com',
'frossi@quo.com']
}

df2 = pd.DataFrame(data2)
df2['DataGroup'] = 'Company_B'
print(df2)

df_concat = pd.concat([df1,df2], ignore_index = True, sort=True)
print(df_concat)
print(df)
```

Describe the data set

df.describe()

Write stats description to csv file step 1 place within a variable df_stats

df_stats = df.describe()

write to variable csv file

df_stats.to_csv("stats.csv",sep=',')

Add a chart to the notebook using matlib Age and Salary

from matplotlib import pyplot as plt

plt.bar(x=df['Age'], height=df['Salary'])

Clear the plot area

plt.clf()

Create a customised bar plot of revenue by Age

plt.bar(x=df['Age'], height=df['Salary'], color='orange')

plt.title('Salary by year')

plt.xlabel('Age')

plt.ylabel('Salary')

plt.grid(color='#95a5a6', linestyle='--', linewidth=2, axis='y', alpha=0.7)

plt.xticks(rotation=45)

plt.show()

Create a figure (fig box) for 2 subplots (1 row, 2 columns)

```
fig, ax = plt.subplots(1, 2, figsize = (10,4))
# Create a bar plot of revenue by Age on the first axis
ax[0].bar(x=df['Age'], height=df['Salary'], color='orange')
ax[0].set_title('Salary by  Age')
# Create a pie chart of yearly order counts on the second axis
yearly_counts = df['Age'].value_counts()
ax[1].pie(yearly_counts)
ax[1].set_title('Rows by Age')
ax[1].legend(yearly_counts.keys().tolist())
# Add a title to the Figure
fig.suptitle('Dataframe Data')
# Show the figure
plt.show()
```

Using seaborn

```
import seaborn as sns

# Clear the plot area

plt.clf()

# Create a bar chart

ax = sns.barplot(x="Age", y="Salary", data=df)

plt.show()

# Clear the plot area

plt.clf()

# Set the visual theme for seaborn

sns.set_theme(style="whitegrid")

 # Create a bar chart

ax = sns.barplot(x="Age", y="Salary", data=df)

plt.show()

# Clear the plot area

plt.clf()

# Create a bar chart

ax = sns.lineplot(x="Age", y="Salary", data=df)

plt.show()
```

Solutions for Python Student Tasks

```
import pandas as pd
import numpy as np

# task 1
df= pd.read_csv("Garage.csv",sep=',')
print(df)

#Task 2
print(df.isnull().sum())

#Task 3
df_filled = df.fillna({'FirstName': 'No Value'})
print(df_filled)

#Task 4
df.replace({'VehicleAge':'Unknown','RepairCost': 'Not Estimated'},np.nan, inplace=True)
print(df)

#Task 6
df['Length_Email'] = df['Email'].str.len()
print(df)

#Task 7
df['BeforeStrip'] = df['Email'].str.len()
df['Email'] = df['Email'].str.strip()
df['AfterStrip'] = df['Email'].str.len()
print(df)
```

```
#Task 8
df['LastName'] = df['LastName'].str.title()
print(df)

#Task 9
df['Date'] = pd.to_datetime(df['Date'])
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['Day'] = df['Date'].dt.day
print(df)


#Task 10
df_stats = df.describe()
print(df_stats)
df_stats.to_csv("stats.csv",sep=',')


#Task 11
import seaborn as sns
# Clear the plot area
plt.clf()
# Create a bar chart
ax = sns.barplot(x="CarValue", y="RepairCost", data=df)
plt.show()
```